

Integración de Aplicaciones Computacionales

Diseño de Bases de Datos

Santiago Lopez Cervantes #612956
Juan Nicolás Núñez Mendoza #613622
Fabian Azaed Orta S. #613504

06/08/2026

Damos nuestra palabra que hemos realizado esta actividad con integridad académica.

ÍNDICE

Diseño de Bases de Datos.....	0
ÍNDICE.....	1
1. Introducción.....	2
2. PostgreSQL.....	2
2.1 Modelo Conceptual.....	3
2.2 Modelo Lógico y Catálogo.....	6
2.3 Modelo Físico.....	9
2.4 Restricciones y Especificaciones Técnicas.....	10
2.5 Datos de Prueba.....	11
2.6 Estrategia de Auditoría y Trazabilidad.....	12
3. MongoDB.....	15
3.1 Colecciones.....	15
3.2 Estructura de Documentos.....	17
3.3 Documentos Embebidos.....	19
3.4 Referencias.....	21
3.5 Índices.....	23
3.6 Versionamiento.....	27
3.7 Estrategia de crecimiento.....	29
4. Redis.....	31
4.1 Estructura de Claves.....	32
4.2 Tiempo de expiración.....	34
4.3 Sesiones.....	35
4.4 Lista de Tokens Revocados en Memoria.....	37
4.5 Caché de Consultas.....	39
4.6 Contadores.....	40
4.7 Bloqueos distribuidos.....	41
4.8 Datos Temporales.....	44
5. Conclusiones.....	45

1. Introducción

Para satisfacer los requerimientos concurrentes de consistencia transaccional, ingesta analítica flexible y baja latencia, el sistema implementa un modelo de persistencia políglota. Esta arquitectura delega cada carga de trabajo al motor de base de datos más adecuado según su naturaleza. Alineando las necesidades de negocio previamente definidas en el [documento analítico](#), el presente documento tiene como finalidad detallar el uso de tecnologías orientadas a las bases de datos para así tener un desglose de división de cargas de trabajo basándonos en la funcionalidad precisa que ofrecen las tecnologías que hemos seleccionado.

2. PostgreSQL

Se ha tomado la decisión de delegar la gestión de accesos, catálogos maestros, trazabilidad de lotes y operación diaria a **PostgreSQL**, tecnología relacional que se fundamenta en las exigencias críticas de un entorno minorista de alto volumen. A continuación, se detallan las razones arquitectónicas.

1) Cumplimiento ACID y Prevención de Condiciones de Carrera (Race Conditions)

El inventario de perecederos es un activo financiero altamente volátil. Cuando múltiples cajeros o gestores de tienda intentan descontar o transferir unidades del mismo lote de manera simultánea, el sistema debe evitar lecturas fantasma o dobles descuentos. PostgreSQL garantiza propiedades **ACID** (Atomicidad, Consistencia, Aislamiento y Durabilidad) y permite aplicar bloqueos a nivel de fila, asegurando que las cantidades disponibles por lote sean matemáticamente exactas en todo momento.

2) Integridad Referencial Estricta

La operación diaria y la auditoría exigen que los datos estén interconectados sin margen de error. PostgreSQL obliga el uso de llaves primarias y foráneas. Esto significa que es imposible registrar la merma de un lote que no existe, asignar una orden de compra a un proveedor no dado de alta, o descontar inventario de una tienda inexistente. Esta rigidez estructural es indispensable para proteger el modelo de **Datos Maestros** y asegurar que el **Control de Accesos** no presente vulnerabilidades por datos huérfanos.

3) Control de Concurrencia

En una cadena de suministro, las operaciones de lectura y las operaciones de escritura ocurren al mismo tiempo. El sistema MVCC de PostgreSQL permite que los lectores no bloqueen a los escritores y viceversa, manteniendo un alto rendimiento operativo sin sacrificar la consistencia del dato consultado.

4) Capacidades Analíticas y Auditoría (Trazabilidad)

Para los requerimientos de **Transacciones y Auditoría**, el lenguaje SQL estandarizado de PostgreSQL ofrece una potencia inigualable en la ejecución de uniones complejas (*JOINS*) y funciones de agregación. Esto facilita enormemente el rastreo histórico de un lote desde su recepción en andén hasta su venta o merma, permitiendo generar dictámenes inalterables y auditorías de trazabilidad inversa en segundos, cumpliendo así con las normativas sanitarias y fiscales.

2.1 Modelo Conceptual

El modelo conceptual es una representación de alto nivel que describe la estructura semántica de la información del negocio. Su objetivo principal es identificar las **Entidades** clave (los "objetos" o conceptos del mundo real) y las **Relaciones** que existen entre ellas, omitiendo detalles técnicos de implementación. Sirve como un puente de comunicación entre las áreas de negocio y el equipo de ingeniería de software, asegurando que todos comprendan *qué* información se va a almacenar antes de decidir *cómo* se programará en el motor de base de datos.

A continuación, se desarrolla el modelo conceptual de las entidades que residirán en el núcleo transaccional de PostgreSQL, agrupadas por su dominio de negocio.

2.1.1 Dominio 1: Seguridad y Control de Acceso

Define quién interactúa con el sistema y bajo qué privilegios.

- **Entidad: Usuario**
 - *Atributos clave:* Identificador, Nombre completo, Correo electrónico, Estado (activo/inactivo).
- **Entidad: Rol**
 - *Atributos clave:* Nombre del rol (ej. Gerente, Comprador), Nivel de autorización.
- **Entidad: Permiso**
 - *Atributos clave:* Acción permitida (ej. "autorizar_merma", "crear_orden").

2.1.2 Dominio 2: Datos Maestros (Master Data)

Son los catálogos base que cambian con poca frecuencia pero alimentan toda la operación.

- **Entidad: Producto (SKU)**
 - *Atributos clave:* Código de barras (GTIN), Nombre, Descripción, Categoría, Unidad de medida, Vida útil estándar, Vida útil mínima de recepción.
- **Entidad: Locación**

- *Atributos clave:* Tipo (Tienda o CEDIS), Nombre, Dirección, Zona geográfica, Capacidad de almacenamiento.
- **Entidad: Proveedor**
 - *Atributos clave:* Razón social, RFC, Tiempo de entrega, Tamaño mínimo de pedido.

2.1.3 Dominio 3: Inventario y Trazabilidad de Lotes

El núcleo logístico donde recae la política de rotación FEFO.

- **Entidad: Lote**
 - *Atributos clave:* Código de lote (asignado por proveedor), Fecha de elaboración, Fecha de caducidad.
- **Entidad: Existencia (Inventario Físico)**
 - *Atributos clave:* Cantidad disponible, Cantidad reservada, Pasillo/Ubicación física en cámara.
- **Entidad: Recepción (Entrada de Mercancía)**
 - *Atributos clave:* Fecha y hora de entrada, Número de factura u orden de compra.

2.1.4 Dominio 4: Transacciones Operativas diarias

Los eventos que alteran el estado del inventario y generan valor o pérdida financiera.

- **Entidad: Venta (Punto de Venta - POS)**
 - *Atributos clave:* Ticket/Folio, Fecha y hora, Precio final pagado, Cantidad vendida.
- **Entidad: Merma (Baja de Inventario)**
 - *Atributos clave:* Fecha de baja, Cantidad mermada, Causa (caducidad, daño).
- **Entidad: Transferencia Intersucursal**
 - *Atributos clave:* Fecha de solicitud, Fecha de envío, Estado (pendiente, en tránsito, recibida).

2.1.5 Mapa de Relaciones Conceptuales

Para entender cómo interactúan estas entidades, se definen las siguientes reglas de cardinalidad y relación.

Usuarios y Locaciones

Un **Usuario** pertenece a una (1) **Locación**, pero una **Locación** puede tener múltiples (N) **Usuarios**. Un **Usuario** tiene asignado un (1) **Rol**, el cual posee múltiples (N) **Permisos**.

Productos, Proveedores y Lotes

Un **Proveedor** suministra múltiples (*N*) **Productos**. Al ingresar mercancía, un **Producto** genera múltiples (*N*) **Lotes** a lo largo del tiempo. Un **Lote** pertenece siempre a un (*1*) único **Producto** y a un (*1*) único **Proveedor**.

Inventario Físico

La **Existencia** es la intersección entre qué hay y dónde está. Una **Existencia** asocia un (*1*) **Lote** específico con una (*1*) **Locación** específica, determinando cuántas unidades de ese lote exacto están en esa tienda en particular.

Ventas y Mermas (Salidas)

Una **Venta** o una **Merma** descuenta unidades siempre de un (*1*) **Lote** específico (para cumplir la trazabilidad FEFO) y ocurre dentro de una (*1*) **Locación**, siendo autorizada o registrada por un (*1*) **Usuario**.

Transferencias

Una **Transferencia** mueve unidades de un (*1*) **Lote** desde una **Locación** de origen hacia una **Locación** de destino.

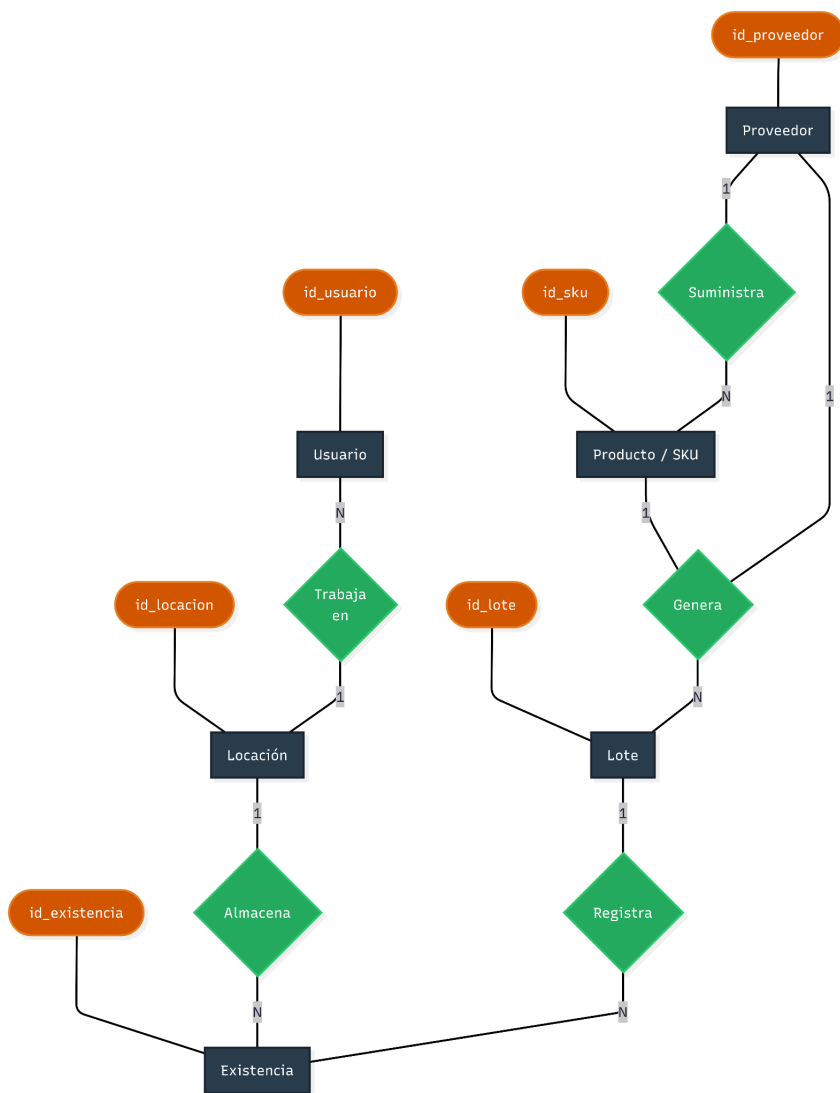
2.2 Modelo Lógico y Catálogo

El Modelo Lógico es el paso de traducción entre los requerimientos de negocio (modelo conceptual) y la implementación técnica final (modelo físico). En esta fase, las abstracciones se transforman en estructuras relacionales concretas. Para esta fase, utilizamos un **Diagrama Lógico en Notación de Chen**, la cual utiliza una representación visual para representar el modelo lógico de la base de datos a desarrollar.

Nota de Diseño:

Para mantener la legibilidad del diagrama y evitar una saturación visual que dificulte su lectura, se han graficado únicamente las entidades principales. El detalle exhaustivo de todos los atributos se encuentra en el diccionario de datos lógico posterior.

2.2.1 Diagrama Conceptual



2.2.2 Diccionario del Modelo Lógico

A continuación, se define la estructura relacional resolviendo las cardinalidades y aplicando tipos de datos estandarizados.

Dominio de Seguridad y Acceso

Entidad / Tabla	Atributo	Tipo de Dato Lógico	Restricción / Llave
Rol	id_rol	Entero Autoincremental	PK
	nombre_rol	Cadena (Varchar)	Único, No Nulo
Usuario	id_usuario	UUID	PK
	id_rol	Entero	FK (Ref: Rol)
	id_locacion	Entero	FK (Ref: Locación)
	nombre_completo	Cadena (Varchar)	No Nulo
	estado_activo	Booleano	Por defecto: Verdadero

Dominio de Datos Maestros

Entidad / Tabla	Atributo	Tipo de Dato Lógico	Restricción / Llave
Locación	id_locacion	Entero Autoincremental	PK
	tipo_locacion	Enum ('Tienda', 'CEDIS')	No Nulo
	nombre	Cadena (Varchar)	No Nulo
Proveedor	id_proveedor	Entero Autoincremental	PK
	rfc	Cadena (13 caracteres)	Único, No Nulo
	razon_social	Cadena (Varchar)	No Nulo
	lead_time_dias	Entero	No Nulo
Producto (SKU)	id_sku	Entero Autoincremental	PK
	codigo_gtin	Cadena (14 caracteres)	Único, No Nulo
	nombre	Cadena (Varchar)	No Nulo
	vida_util_estandar	Entero (días)	No Nulo

Dominio de Inventario y Trazabilidad

*Nota: La entidad **Existencia** funciona como tabla asociativa resolviendo la relación N:M entre **Lote** y **Locación**.*

Entidad / Tabla	Atributo	Tipo de Dato Lógico	Restricción / Llave
Lote	id_lote	UUID	PK
	id_sku	Entero	FK (Ref: Producto)
	id_proveedor	Entero	FK (Ref: Proveedor)
	codigo_lote_prov	Cadena (Varchar)	No Nulo
	fecha_caducidad	Fecha (Date)	No Nulo
Existencia	id_existencia	UUID	PK
	id_lote	UUID	FK (Ref: Lote)
	id_locacion	Entero	FK (Ref: Locación)
	cantidad_disponible	Decimal / Numérico	No Nulo, Mayor a 0
	cantidad_reservada	Decimal / Numérico	Por defecto: 0

Dominio de Transacciones Diarias

Entidad / Tabla	Atributo	Tipo de Dato Lógico	Restricción / Llave
Venta_Detalle	id_venta	UUID	PK
	id_lote	UUID	FK (Ref: Lote)
	id_locacion	Entero	FK (Ref: Locación)
	cantidad	Decimal / Numérico	No Nulo
	fecha_transaccion	Fecha y Hora (Timestamp)	No Nulo
Merma	id_merma	UUID	PK
	id_lote	UUID	FK (Ref: Lote)
	id_usuario	UUID	FK (Ref: Usuario)

	cantidad	Decimal / Numérico	No Nulo
	causa_merma	Enum ('Caducidad', 'Daño')	No Nulo

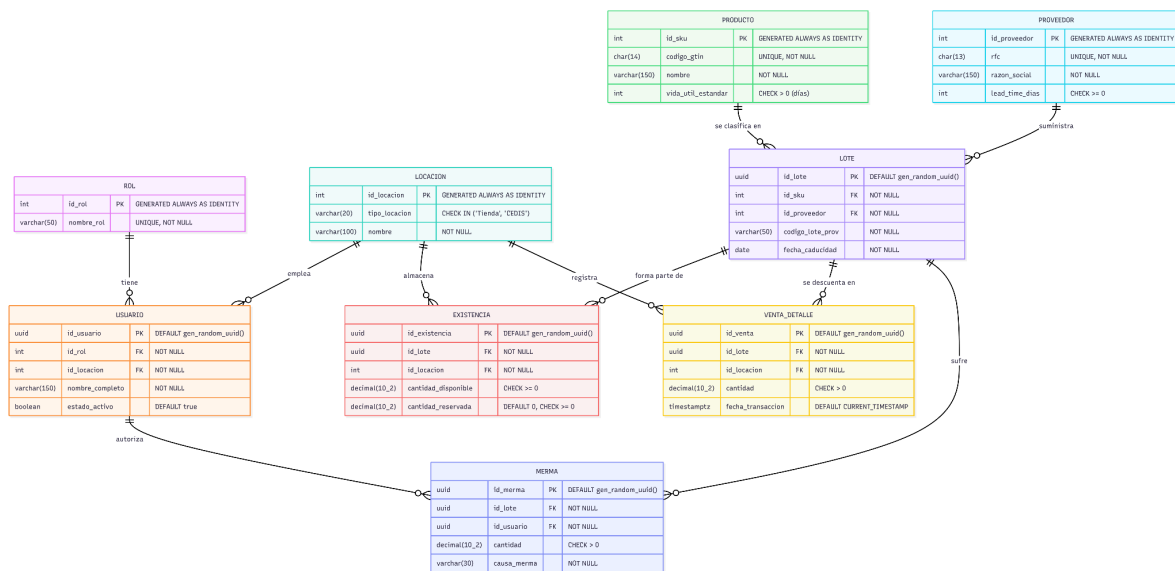
2.3 Modelo Físico

El Modelo Físico representa la implementación técnica final de la base de datos, adaptando las abstracciones del modelo lógico a las características, tipos de datos y sintaxis específicas del Sistema Gestor de Bases de Datos (RDBMS) seleccionado, en este caso **PostgreSQL**.

A diferencia del modelo lógico, el diseño físico define con exactitud la capacidad de almacenamiento (e.g., VARCHAR(50) vs. TEXT), los motores generadores de identificadores (e.g., UUID, SERIAL), y las restricciones de integridad y dominio operativas a nivel de base de datos (CHECK, DEFAULT, UNIQUE), las cuales garantizan que la información corrupta no pueda ingresar al sistema ni siquiera por errores en el código de la aplicación.

A continuación se presenta el diagrama físico utilizando la notación **Crow's Foot**, la cual es el estándar en ingeniería de software para ilustrar la cardinalidad exacta, las llaves primarias (PK) y las llaves foráneas (FK).

2.3.1 Diagrama Físico



2.4 Restricciones y Especificaciones Técnicas

Con el objetivo de garantizar la integridad y el rendimiento del sistema, el modelo físico incluye las siguientes consideraciones de implementación.

2.4.1 Tipos de Datos e Identificadores

UUIDv4 (gen_random_uuid())

Utilizado para registros de alta transaccionalidad (USUARIO, LOTE, EXISTENCIA, VENTA_DETALLE, MERMA). Evita colisiones en sistemas distribuidos y previene que un atacante o competidor deduzca el volumen de ventas o mermas adivinando identificadores secuenciales.

Secuencias de Identidad (INT GENERATED ALWAYS AS IDENTITY)

Utilizado en catálogos de bajo volumen y lectura intensiva (ROL, LOCACION, PRODUCTO, PROVEEDOR). Son más eficientes para llaves foráneas y JOINS que los UUIDs y reemplazan al antiguo tipo SERIAL de Postgres por ser el estándar moderno de SQL.

Zonas Horarias (TIMESTAMPTZ)

Para VENTA_DETALLE, usar TIMESTAMP WITH TIME ZONE asegura que las ventas registradas en tiendas de distintas zonas geográficas de México se normalicen correctamente en UTC para el análisis de los pronósticos.

2.4.1 Restricciones de Dominio

Las restricciones a nivel de tabla bloquean anomalías lógicas que podrían colapsar los algoritmos de pronóstico y reabastecimiento.

Inventario Negativo

Se aplica CHECK (cantidad_disponible $\geq$ 0) en la tabla EXISTENCIA. Si una transacción de venta intenta descontar más producto del que existe en ese lote, la base de datos arrojará un error de restricción, forzando a la aplicación a manejar la falta de sincronización del inventario físico.

Cantidades Lógicas

En VENTA_DETALLE y MERMA, se exige CHECK (cantidad $>$ 0). No se pueden registrar ventas o mermas en ceros o negativas.

Valores Enumerable/Fijos

En LOCACION, la columna tipo_locacion tiene un CHECK IN ('Tienda', 'CEDIS') para evitar inserciones de texto basura.

2.4.3 Restricciones de Integridad Referencial (Foreign Keys)

Las reglas de borrado para catálogos maestros (PRODUCTO, PROVEEDOR) deben configurarse con ON DELETE RESTRICT. Esto impide borrar un proveedor o producto si existen lotes o mermas asociadas, manteniendo intacta la pista de auditoría financiera e histórica.

2.5 Datos de Prueba

El poblamiento de la base de datos con datos de prueba es una fase crítica en la ingeniería de software y la integración de sistemas. En etapas tempranas, una base de datos vacía no permite validar la eficiencia de las consultas, ni el comportamiento de las interfaces, ni la robustez de las reglas de negocio. Por ello, la inyección de un volumen representativo de datos controlados permite simular el comportamiento en producción, someter a estrés la arquitectura relacional y certificar que la integración con los componentes externos opera bajo los estándares requeridos.

A continuación, se detalla la estrategia de poblamiento de datos orientada exclusivamente al motor transaccional PostgreSQL.

2.5.1 Fuentes de Obtención de Datos

Dado que los datos de un comercio minorista real suelen estar protegidos por acuerdos de confidencialidad, los datos de prueba se obtendrán mediante un enfoque híbrido:

Generación Sintética Estocástica (Scripts Propios)

Se desarrollarán *scripts* de generación de datos (utilizando lenguajes como Python con librerías como **Faker** y **Pandas**) para poblar los catálogos de Usuarios, Proveedores y Locaciones. Estos *scripts* insertarán datos aleatorios pero semánticamente correctos (ej. correos válidos, RFCs con formato legal, nombres coherentes).

Adaptación de Datasets Públicos de Retail

Para los catálogos de **Productos (SKUs)** y las transacciones de **Ventas**, se utilizarán *datasets* abiertos de la industria (como los proporcionados en competencias de *Kaggle* tipo *M5 Forecasting* o históricos anonimizados de supermercados). Estos datos crudos se transformarán mediante procesos ETL (*Extract, Transform, Load*) para mapearlos y normalizarlos de acuerdo con el esquema físico y las restricciones definidas en nuestro diagrama *Crow's Foot*.

2.5.1 Volumen Esperado de Datos

Para que las pruebas de integración y rendimiento sean válidas, la base de datos debe manejar un volumen que simule al menos **6 meses de operación continua**. Se proyecta la siguiente volumetría.

Datos Maestros

- ~50 a 100 perfiles de **USUARIO**.
- ~20 a 50 **LOCACIONES** (distribuidas entre Tiendas y CEDIS).
- ~500 **PRODUCTOS** (SKUs limitados a la categoría de perecederos).
- ~30 a 50 **PROVEEDORES**.

Datos Operativos y Transaccionales

- **Lotes:** ~50,000 registros (simulando múltiples entregas semanales por producto).
- **Existencias:** ~150,000 registros (la intersección de los lotes distribuidos entre las distintas locaciones).
- **Venta Detalle:** > 500,000 registros (simulando el goteo constante de ventas en el Punto de Venta).
- **Mermas:** ~15,000 registros (simulando una tasa de pérdida controlada del 2% al 4%).

2.6 Estrategia de Auditoría y Trazabilidad

En un sistema transaccional que gestiona activos financieros volátiles (inventario perecedero) y reporta métricas con impacto fiscal y normativo (mermas deducibles ante el SAT, cumplimiento sanitario ante COFEPRIS y métricas ESG por desperdicio evitado), la auditoría no puede depender exclusivamente de los registros de la aplicación. Si la aplicación sufre una vulnerabilidad o un error lógico, la historia de los datos podría corromperse.

Por lo tanto, la estrategia de auditoría se implementará a nivel de base de datos dentro de **PostgreSQL**, garantizando que cualquier modificación a los datos operativos deje una huella inalterable, independientemente de la interfaz o API que haya originado el cambio.

2.6.1 Auditoría a Nivel de Fila

Para evitar la proliferación de tablas de relación débil, se implementará una tabla de bitácora centralizada utilizando el tipo de dato **JSONB** nativo de PostgreSQL. De esta manera, se programarán disparadores (*Triggers*) que se ejecutarán automáticamente de forma subordinada a los eventos **INSERT**, **UPDATE** y **DELETE** sobre las tablas críticas (**USUARIO**, **EXISTENCIA**, **MERMA**, **VENTA_DETALLE**).

Cada vez que una fila cambie, el *trigger* insertará un nuevo registro en la tabla **esquema_auditoria.bitacora_eventos** capturando:

- **nombre_tabla:** La tabla afectada.
- **tipo_operacion:** Inserción, actualización o borrado.
- **id_usuario:** El UUID del usuario que ejecutó la transacción.

- **estado_anterior**: Un objeto *JSONB* con los datos exactos antes de la modificación.
- **estado_nuevo**: Un objeto *JSONB* con los datos resultantes.
- **fecha_evento**: Estampa de tiempo exacta (Timestamp UTC).

2.6.2 Inmutabilidad de los Datos (*Modelo Append-Only*)

La tabla de auditoría debe ser criptográfica o lógicamente inalterable para tener validez.

Restricción de Privilegios

A nivel del catálogo del sistema, el rol de conexión utilizado por el backend de la aplicación web solo tendrá permisos de INSERT y SELECT sobre el esquema de auditoría. Se le denegarán de forma absoluta los comandos UPDATE, DELETE y TRUNCATE.

Protección contra el Administrador

Incluso si un perfil con privilegios elevados (Nivel 5) intenta alterar la bitácora, la estructura del sistema obligará a que dicho cambio quede registrado en los logs a nivel de servidor físico, asegurando la segregación de funciones.

2.6.3 Extensión *pgAudit* para Cumplimiento Normativo Severo

Para complementar los triggers (que registran el "qué" cambió), el sistema habilitará la extensión nativa **pgAudit** (*PostgreSQL Audit Extension*).

- **Objetivo**: pgAudit no guarda los datos en tablas, sino que registra las sentencias SQL exactas directamente en los archivos log del sistema operativo del servidor.
- **Beneficio**: Es un estándar de la industria requerido para auditorías estrictas (como SOC 2 o ISO 27001). Permite reconstruir eventos catastróficos, rastrear consultas de solo lectura sensibles (ej. quién consultó el catálogo completo de proveedores) y detectar intentos de inyección SQL que hayan llegado al motor.

3. MongoDB

MongoDB es un sistema gestor de bases de datos orientado a documentos NoSQL que almacena la información en estructuras flexibles similares a JSON (BSON). A diferencia del modelo relacional estricto de PostgreSQL, MongoDB permite modelar registros polimórficos, esquemas dinámicos y anidamiento de estructuras complejas sin requerir migraciones de esquema cada vez que evoluciona el modelo de datos. En este proyecto, la capa documental desempeña un rol complementario y especializado.

Ingesta de datos no estructurados y cambiantes

Las inferencias de modelos de Machine Learning (ML) y las alertas logísticas tipo ASN (Advanced Shipping Notice) contienen metadatos heterogéneos, intervalos de confianza variables y estructuras que cambian con cada iteración del pipeline analítico.

Desacoplamiento de la transaccionalidad operativa

Separa las cargas de trabajo analíticas y de series temporales del motor OLTP principal (PostgreSQL), evitando bloqueos y contención de recursos en las tablas de ventas, mermas e inventarios.

Lecturas agregadas de alto rendimiento

Facilita consultas embebidas donde los pronósticos de demanda, fechas de caducidad proyectadas y avisos de despacho se consultan como documentos atómicos, acelerando la toma de decisiones sin necesidad de costosos JOINS relacionales en tiempo real.

3.1 Colecciones

La capa NoSQL está estructurada para gestionar cargas analíticas, series de tiempo predictivas y eventos de comunicación externa con la cadena de suministro, operando bajo un esquema desacoplado de la base de datos relacional.

3.1.1 Colección: *predicciones_demanda*

Almacena las salidas de los modelos de Machine Learning (pronósticos de ventas, riesgo de merma y recomendaciones de reabastecimiento). Maneja la estructura temporal de predicción y las métricas de incertidumbre del modelo.

Las salidas de los modelos analíticos contienen hiperparámetros, rangos probabilísticos (intervalos de confianza) y versionado de algoritmos que cambian frecuentemente. Un esquema BSON permite guardar estas inferencias sin alterar estructuras de tablas relacionales.

Componentes clave de información:

- Identificadores de negocio foráneos (sku_id, locacion_id) para trazabilidad cruzada con el esquema relacional.

- Ventana de predicción (fecha de corte, horizonte temporal proyectado).
- Salida analítica (demanda estimada puntual, límites superior/inferior de tolerancia, probabilidad calculada de merma).
- Metadatos del pipeline (versión del artefacto o modelo ML, identificador de la corrida de inferencia, métricas de confianza).

3.1.2 Colección: avisos_asn

Registra los avisos anticipados de embarque emitidos por proveedores y centros de distribución (CEDIS) antes de la recepción física de mercancía perecedera.

Los manifiestos de envío varían significativamente según el formato de intercambio electrónico de datos (EDI), número de bultos, condiciones de transporte y subdivisiones de lotes por proveedor. El anidamiento jerárquico permite empaquetar cabeceras y detalles logísticos en un único documento atómico.

Componentes clave de información:

- Identificador externo del envío (número de guía, ID de orden de compra origen).
- Contexto de origen/destino (identificador del proveedor emisor, locación o tienda receptora).
- Fechas operativas (fecha de despacho programada, ventana estimada de arribo al muelle).
- Documentos embebidos de carga (listado de SKUs, cantidades declaradas, números de lote del fabricante y fechas de vencimiento reportadas).
- Condiciones de cadena de frío y transporte (temperatura declarada de transporte, tipo de vehículo, estatus de despacho).

3.1.3 Colección: alertas_operativas

Centralizar advertencias automatizadas generadas a partir del cruce de inventario actual, pronósticos de demanda y desviaciones detectadas en los envíos ASN.

Permite la ingesta en ráfaga de eventos transitorios con diversos niveles de severidad y ciclo de vida corto, facilitando el uso de índices de expiración (TTL) para depuración automática.

Componentes clave de información:

- Nivel de criticidad (informativa, advertencia, bloqueo crítico).
- Tipo de evento (riesgo de caducidad por política FEFO, retraso de entrega logística, discrepancia de inventario vs. pronóstico).
- Referencias vinculadas (lote afectado, tienda/CEDIS involucrado, código de producto).
- Estado de resolución y marca temporal de generación.

3.2 Estructura de Documentos

A continuación se detalla la arquitectura de campos y tipos de datos para cada colección documental, modelada mediante estructuras jerárquicas en árbol.

3.2.1 Colección: *pronosticos_demanda*

Diseñada para persistir la inferencia analítica con granularidad diaria por producto y locación, integrando intervalos probabilísticos y trazabilidad del modelo.

```
None
pronosticos_demanda/
├── _id: ObjectId          # Clave primaria documental
├── sku_id: Integer        # Referencia foránea a operacion.producto.id_sku
├── locacion_id: Integer   # Referencia foránea a
operacion.locacion.id_locacion
├── fecha_ejecucion: ISODate # Timestamp exacto de generación de la
inferencia
├── horizonte_proyeccion/  # Ventana temporal analizada
│   ├── fecha_inicio: ISODate # Inicio del periodo proyectado
│   └── fecha_fin: ISODate    # Término del periodo proyectado
├── prediccion/           # Salidas numéricas del estimador
│   ├── demanda_esperada_unidades: Double # Valor puntual proyectado
│   ├── intervalo_confianza_95/          # Rango de dispersión probabilística
│   │   ├── limite_inferior: Double      # Escenario pesimista / cota mínima
│   │   └── limite_superior: Double      # Escenario optimista / cota máxima
│   └── probabilidad_merma: Double       # Score de propensión a pérdida (0.00 a
1.00)
├── merma_estimada_unidades: Double # Volumen proyectado en riesgo
├── metadata_modelo/       # Linaje y gobierno del pipeline analítico
│   └── modelo_id: String   # Identificador del artefacto (ej.
"xgboost_perishables")
│       ├── version: String # Versión semántica del modelo (ej. "v2.1.0")
│       └── metrica_mae_historica: Double # Error medio absoluto de referencia del
artefacto
```

3.2.2 Colección: *avisos_asn*

Estructura orientada a digitalizar manifiestos anticipados de transporte, desacoplando la cabecera operativa y embebiendo los detalles de lotes y cadena de frío en una sola unidad de lectura.

None

```
avisos_asn/  
├── _id: ObjectId          # Clave primaria documental  
├── numero_asn: String      # Manifiesto/guía externa única (ej.  
"ASN-2026-00892")  
├── orden_compra_origen: String # Referencia contable a la orden de compra  
├── proveedor_id: Integer    # Referencia foránea a  
operacion.proveedor.id_proveedor  
├── locacion_destino_id: Integer # Referencia foránea a  
operacion.locacion.id_locacion  
├── estado_recepcion: String  # Estado ("EN_TRANSITO", "RECIBIDO",  
"DISCREPANCIA")  
├── logistica_transporte/    # Condiciones físicas de despacho  
│   ├── transportista: String # Razón social del operador logístico  
│   ├── tipo_unidad: String   # Vehículo ("Caja Refrigerada", "Camión Rabón")  
│   ├── placas: String        # Identificador vehicular  
│   └── rango_temperatura_controlada/ # Monitoreo de cadena de frío (°C)  
│       ├── temp_minima: Double # Umbral inferior admitido  
│       └── temp_maxima: Double # Umbral superior admitido  
├── cronograma/              # Hitos temporales de entrega  
│   ├── fecha_despacho: ISODate # Salida física del centro emisor  
│   └── ventana_estimada_arribo: ISODate # Ventana comprometida en muelle  
└── detalle_carga: Array<Object> # Manifiesto desagregado de productos  
    └── [item]/  
        ├── sku_id: Integer    # Referencia foránea a operacion.producto.id_sku  
        ├── lote_proveedor: String # Código de lote emitido por fábrica  
        ├── cantidad_embarcada: Double # Total de unidades declaradas  
        └── fecha_caducidad_origen: Date # Fecha de vencimiento declarada para  
FEFO
```

3.2.3 Colección: *alertas_operativas*

Estructura ligera orientada a eventos, diseñada para soportar altas tasas de escritura de advertencias operativas y mecanismos automáticos de expiración de ciclo de vida (TTL).

None

```
alertas_operativas/  
├── _id: ObjectId          # Clave primaria documental  
├── tipo_alerta: String     # Clasificador ("RIESGO_CADUCIDAD_FEFO",  
"DESVIACION_ASN")
```

— nivel_severidad: String	# Criticidad ("INFO", "WARNING", "CRITICAL")
— descripcion: String	# Mensaje procesable para el personal de tienda/CEDIS
— entidad_afectada/relacional	# Cruzamiento contextual con el modelo relacional
— locacion_id: Integer	# Tienda o CEDIS impactado
— sku_id: Integer	# Producto involucrado
— lote_id: UUID	# Referencia foránea a operacion.lote.id_lote
(opcional)	
— estado: String	# Estatus operativo ("PENDIENTE", "ATENDIDO", "IGNORADO")
— fecha_generacion: ISODate	# Timestamp de creación del evento
— expira_en: ISODate	# Índice TTL para depuración automática en MongoDB

3.3 Documentos Embebidos

En el diseño documental sobre MongoDB para la gestión de perecederos, la técnica de embebido (*denormalization / embedding*) se aplica selectivamente para garantizar atomicidad transaccional, eliminación de operaciones de unión (*JOINS*) y localidad de referencia en lecturas frecuentes.

A continuación se definen los criterios de diseño y las estructuras embebidas implementadas en el repositorio analítico-logístico.

3.3.1 Criterios de Selección: Embebido vs. Referencia

La decisión de embeber subdocumentos se rige bajo los siguientes principios arquitectónicos:

Relación de contención estricta (1:1 o 1:Pocos acotado)

La información no tiene ciclo de vida independiente fuera del documento padre (e.g., los rangos de un intervalo de confianza no existen sin su predicción).

Consistencia atómica en escritura

Toda la operación se persiste en una sola instrucción atómica en el motor BSON, eliminando transacciones distribuidas o bloqueos en múltiples colecciones.

Patrón de acceso conjunto

La aplicación siempre consume la cabecera y el detalle en la misma consulta (e.g., al consultar un manifiesto ASN en muelle de recepción, se leen el transporte y las líneas de producto en un único *fetch*).

3.3.2 Detalle de Estructuras Embebidas por Colección

1. En `pronosticos_demanda`

- **Objeto `horizonte_proyeccion` (1:1)**
 - **Estructura:** { `fecha_inicio`: ISODate, `fecha_fin`: ISODate }
 - Representa la delimitación temporal estricta de la inferencia. Embeber estos campos encapsula el intervalo de evaluación en una sola unidad semántica, facilitando consultas de rango compuestas (`horizonte_proyeccion.fecha_inicio`).
- **Objeto `prediccion` y subdocumento `intervalo_confianza_95` (1:1 jerárquico)**
 - **Estructura:** Contiene métricas de salida (`demanda_esperada_unidades`, `probabilidad_merma`) y el objeto anidado { `limite_inferior`: Double, `limite_superior`: Double }.
 - Los valores proyectados y sus márgenes de error son inmutables tras la ejecución del pipeline. Embeberlos evita crear colecciones satélite de dispersión estadística y reduce la latencia en paneles analíticos.
- **Objeto `metadata_modelo` (1:1)**
 - **Estructura:** { `modelo_id`: String, `version`: String, `metrica_mae_historica`: Double }
 - Asegura el linaje del dato. Si el modelo se recalibra o se entrena una nueva versión, el pronóstico previo conserva una copia estática de la versión y precisión con la que fue generado (trazabilidad inmutable).

2. En `aviso_asn`

- **Objeto `logistica_transporte` y subdocumento `rango_temperatura_controlada` (1:1)**
 - **Estructura:** Parámetros de transportista, vehículo y el rango { `temp_minima`: Double, `temp_maxima`: Double }.
 - La cadena de frío y el transporte describen exclusivamente el trayecto de ese envío en particular. Mantenerlo embebido permite al personal en andén validar inmediatamente si el termógrafo del camión coincide con el rango admitido.
- **Arreglo `detalle_carga` (1:N acotado)**
 - **Estructura:** Array<Object> donde cada elemento agrupa { `sku_id`: Integer, `lote_proveedor`: String, `cantidad_embarcada`: Double, `fecha_caducidad_origen`: Date }.
 - Un envío típico de perecederos a tienda o CEDIS contiene entre 10 y 150 renglones. Este volumen está muy por debajo del límite de 16 MB por documento de BSON. Agrupar la carga completa en un arreglo permite:
 1. Confirmar la recepción en muelle en un único viaje de red (*round-trip*).

2. Comparar en memoria la cantidad declarada contra la cantidad leída por radiofrecuencia (RFID) o código de barras.

3. En alertas_operativas

- **Objeto entidad_afectada (1:1)**
 - **Estructura:** { locacion_id: Integer, sku_id: Integer, lote_id: UUID }
 - Agrupa las claves foráneas que contextualizan la anomalía sin requerir un esquema normalizado. Permite indexar de forma compuesta subcampos (entidad_afectada.locacion_id, entidad_afectada.sku_id) para filtrar todas las alertas activas de una tienda específica en tiempo constante $O(1)$.

3.3.3 Impacto en Índices y Rendimiento

El uso de subdocumentos embebidos optimiza la indexación en MongoDB:

- **Notación de puntos (*dot notation*):** Se construyen índices sobre rutas internas sin aplanar el documento (e.g., db.pronosticos_demanda.createIndex({ "horizonte_proyeccion.fecha_inicio": 1, "sku_id": 1 })).
- **Índices Multikey:** Permite indexar campos dentro del arreglo de carga (e.g., db.avisos_asn.createIndex({ "detalle_carga.sku_id": 1 })), lo que agiliza la búsqueda de cualquier embarque en tránsito que contenga un SKU en riesgo de desabasto.

3.4 Referencias

A diferencia del paradigma de documentos embebidos (utilizado para datos atómicos y acotados), el modelo de *referencias* en esta arquitectura se implementa para articular la persistencia políglota: conecta los documentos semiestructurados de MongoDB con el modelo transaccional normalizado de PostgreSQL.

3.4.1 Principios de Decisión: ¿Cuándo Referenciar?

La vinculación por identificador foráneo se adopta bajo los siguientes criterios técnicos:

- **Datos maestros altamente normalizados:** Entidades como proveedores, locaciones (tiendas/CEDIS) y productos cambian sus atributos principales (e.g., razón social, lead time, directores de sucursal) de manera independiente. Duplicar esta información en documentos generaría anomalías de actualización.
- **Trazabilidad cruzada sin acoplamiento de almacenamiento:** Permite a los pipelines analíticos y microservicios operar con IDs compactos (sku_id, locacion_id, lote_id), delegando las consultas detalladas de catálogos al motor OLTP relacional únicamente cuando sea necesario.
- **Prevención del límite de tamaño BSON:** Evita el antipatrón de crecimiento desmedido (*unbounded growth*) al desacoplar entidades operativas que tienen un ciclo de vida propio.

3.4.2 Mapeo de Identificadores Cruzados (PostgreSQL - MongoDB)

Colección MongoDB	Campo Foráneo en Documento	Tipo de Dato	Tabla / Columna Relacional Destino	Propósito del Enlace
pronosticos_demanda	sku_id	Integer	operacion.producto (id_sku)	Cruzar el pronóstico de demanda con el nombre del producto y vida útil estándar.
pronosticos_demanda	locacion_id	Integer	operacion.locacion (id_locacion)	Identificar la tienda o CEDIS donde se proyecta el consumo o la merma.
avisos_asn	proveedor_id	Integer	operacion.proveedor (id_proveedor)	Validar términos comerciales, RFC y lead time contractual del remitente.
avisos_asn	locacion_destino_id	Integer	operacion.locacion (id_locacion)	Asignar la recepción física al muelle correspondiente.
avisos_asn	detalle_carga.[item].sku_id	Integer	operacion.producto (id_sku)	Verificar qué artículos vienen dentro del embarque antes de ingresar al inventario.
alertas_operativas	entidad_afectada.locacion_id	Integer	operacion.locacion (id_locacion)	Filtrar alertas operativas en tableros por sucursal específica.

alertas_operativas	entidad_afectada.sku_id	Integer	operacion.producto (id_sku)	Focalizar desviaciones de abastecimiento o riesgo de merma por artículo.
alertas_operativas	entidad_afectada.lote_id	UUID (String BSON)	operacion.lote (id_lote)	Vincular la alerta directamente al registro auditable del lote para la política FEFO.

3.4.3 Patrón de Integración Operativa

1. Resolución en Capa de Aplicación / Backend:

- El servicio backend consulta MongoDB para recuperar los pronósticos y alertas operativas de un centro o producto en milisegundos.
- Si la interfaz de usuario requiere desplegar descripciones enriquecidas (e.g., nombre comercial del proveedor o código de barras GTIN), resuelve las referencias ejecutando una consulta por lote (WHERE id IN (...)) sobre PostgreSQL utilizando los IDs referenciados.

2. Indexación de Claves Foráneas en MongoDB:

- Para evitar escaneos de colección completa (*collection scans*) al cruzar información, cada clave referenciada cuenta con índices dedicados o compuestos:
 - pronosticos_demanda: Índice compuesto { "locacion_id": 1, "sku_id": 1, "fecha_ejecucion": -1 }
 - avisos_asn: Índice { "proveedor_id": 1, "estado_recepcion": 1 }
 - alertas_operativas: Índice compuesto { "entidad_afectada.locacion_id": 1, "estado": 1 }

3. Manejo de UUIDs Relacionales:

- Las claves primarias relacionales generadas mediante `gen_random_uuid()` (como `operacion.lote.id_lote`) se preservan como cadenas alfanuméricas canónicas (UUIDv4) en los subdocumentos de MongoDB, garantizando compatibilidad binaria estricta entre ambos motores de base de datos.

3.5 Índices

3.5.1 Colección: *pronosticos_demanda*

Las consultas más frecuentes involucran recuperar la predicción más reciente generada para un producto en una sucursal específica, o filtrar ventanas temporales completas para tableros analíticos.

Nombre del Índice	Claves / Campos Indexados	Tipo de Índice	Justificación Técnica y Caso de Uso
idx_locacion_sku_fecha	{"locacion_id": 1, "sku_id": 1, "fecha_ejecucion": -1}	Compuesto	Consulta principal del backend: Permite buscar los pronósticos de un producto en una tienda y ordenar de inmediato por la ejecución más reciente en $O(\log N)$, aplicando la regla de igualdad, ordenación y rango (<i>ESR</i>).
idx_horizonte_inicio	{"horizonte_proyeccion.fecha_inicio": 1}	Simple (Dot Notation)	Filtro de vigencia: Permite a los procesos por lote (<i>batch</i>) consultar todas las proyecciones activas a partir de la fecha de inicio del ciclo operativo.
idx_probabilidad_mermas	{"prediccion.probabilidad_merma": -1}	Simple (Dot Notation)	Detección preventiva: Acelera la generación de reportes ejecutivos que priorizan los productos perecederos con mayor riesgo inminente de descomposición.

3.5.2 Colección: *avisos_asn*

El andén de recepción y el área de tráfico requieren validar rápidamente la existencia de guías, identificar despachos en ruta y localizar embarques que contengan lotes o productos específicos.

Nombre del Índice	Claves / Campos Indexados	Tipo de Índice	Justificación Técnica y Caso de Uso
uq_numero_asn	{"numero_asn": 1}	Único	Integridad de negocio: Garantiza que no existan manifiestos o números de guía duplicados emitidos por los proveedores o el sistema EDI.
idx_destino_estado_arribo	{"locacion_destino_id": 1, "estado_recepcion": 1, "cronograma.ventana_estimada_arribo": 1}	Compuesto	Operación en muelle (Andén): Permite a los CEDIS y tiendas listar en tiempo real los camiones clasificados como EN_TRANSITO ordenados cronológicamente por su hora estimada de llegada.
idx_multikey_carga_sku	{"detalle_carga.sku_id": 1}	Multikey	Rastreo de inventario en tránsito: Facilita identificar qué despachos en camino transportan un SKU específico cuando se reporta desabasto crítico en tienda.

idx_multikey_carga_caducidad	{"detalle_carga.fecha_caducidad_origen": 1}	Multikey	Auditoría de política FEFO anticipada: Permite detectar con anticipación lotes que vengan de fábrica con vida útil residual inferior a los umbrales mínimos contractuales.
------------------------------	---	----------	--

3.5.3 Colección: *alertas_operativas*

Esta colección opera con un ciclo de vida volátil; requiere filtrado eficiente por gravedad/tienda y un mecanismo automático de descarte para no sobrecargar el almacenamiento.

Nombre del Índice	Claves / Campos Indexados	Tipo de Índice	Propiedades / TTL	Justificación Técnica y Caso de Uso
idx_locacion_estado_severidad	{"entidad_afectada.locacion_id": 1, "estado": 1, "nivel_severidad": 1}	Compuesto	Estándar	Bandeja de alertas: Alimenta los monitores de incidencias operativas en tienda, permitiendo filtrar incidencias no resueltas (estado: "PENDIENTE") ordenadas por nivel de severidad.
ttl_expiracion_alerta	{"expira_en": 1}	TTL (Time-To-Live)	expireAfter Seconds: 0	Autodepuración: MongoDB elimina físicamente los

				documentos de alerta cuando la hora del sistema alcanza el valor del campo <code>expira_en</code> , evitando la necesidad de tareas programadas tipo <i>cron</i> para mantenimiento.
<code>idx_alerta_lote</code>	<code>{"entidad_afectada.lote_id": 1}</code>	Simple	Parcial (sparse)	Trazabilidad de anomalías: Permite consultar de forma ágil el historial de advertencias generadas en torno a un lote relacional específico.

3.5.4 Consideraciones de Mantenimiento y Memoria

- **Alineación con memoria RAM:** El tamaño acumulado de los índices está proyectado para residir permanentemente en memoria RAM de MongoDB, garantizando lecturas con acceso indexado sin recurrir a lecturas de disco.
- **Sobrecarga de escritura:** En `aviso_asn`, el índice *Multikey* se limita exclusivamente a `sku_id` y `fecha_caducidad_origen` dentro del arreglo `detalle_carga` para no penalizar el rendimiento durante la ingesta masiva de manifiestos.

3.6 Versionamiento

A diferencia de los entornos relacionales donde las migraciones implican bloqueos y sentencias DDL pesadas (`ALTER TABLE`), en MongoDB la evolución del modelo de datos se gestiona a nivel de aplicación y metadatos documentales. Esto permite introducir cambios estructurales sin tiempo de inactividad (*zero-downtime*) y mantener compatibilidad hacia atrás en pipelines de Machine Learning y logística.

Se implementa una estrategia de versionamiento en tres niveles: *Esquema Documental*, *Modelos Analíticos (ML)* y *Evolución de Manifiestos (ASN)*.

3.6.1 Patrón de Versionamiento de Esquema (Schema Versioning Pattern)

Cada documento persistido en las colecciones de MongoDB incorpora un campo de control de versión a nivel raíz:

- **Campo identificador:** `schema_version` (Integer o String semántico, e.g., 1, 2 o "1.0").
- **Propósito:** Permite que la capa de aplicación identifique instantáneamente la estructura del documento recuperado e invoque el adaptador o serializador adecuado.

Mecanismo de Mutación Polimórfica

- **Lectura adaptativa:** Si la aplicación consulta un documento con `schema_version`: 1 pero el código ya opera en la versión 2, una capa de transformación en memoria añade los campos por defecto o reestructura las claves antes de entregar el objeto a la lógica de negocio.
- **Migración perezosa (Lazy Migration):** La base de datos no se actualiza masivamente. La migración física de la versión 1 a la versión 2 ocurre únicamente cuando un documento existente se lee, se modifica y se vuelve a guardar (*read-modify-write*).
- **Migración en lote controlada (Background Batch Migration):** Para colecciones con TTL o documentos históricos que no se actualizan con frecuencia, un script en segundo plano procesa los registros obsoletos en bloques pequeños para no saturar el working set ni degradar el I/O del motor.

3.6.2 Versionamiento de Artefactos e Inferencias de Machine Learning

En la colección `pronosticos_demanda`, el versionamiento no solo aplica a la estructura BSON, sino a la trazabilidad y linaje algorítmico de las predicciones.

Atributo en metadata_modelo	Tipo	Función en el Versionamiento
modelo_id	String	Identificador del algoritmo o arquitectura (ej. <code>xgboost_perishables_daily</code>).
version	String	Versión semántica del artefacto serializado (ej. <code>v1.4.2</code>), vinculada al registro de experimentos.

fecha_entrenamiento	ISODate	Momento exacto en que los pesos o hiperparámetros del modelo fueron ajustados.
pipeline_run_id	String	Hash del commit de Git o identificador de ejecución del pipeline de datos que generó las proyecciones.

Regla de Negocio

Cuando se despliega un modelo con una arquitectura radicalmente distinta (ej. migración de un modelo tabular XGBoost a una red neuronal recurrente para series de tiempo), el valor de version se incrementa formalmente (v1.x - v2.0). Esto permite coexistencia en producción: el backend puede contrastar inferencias del modelo anterior y del nuevo modelo en pruebas A/B sin corromper la colección.

3.6.3 Versionamiento de Manifiestos Logísticos (ASN)

Los avisos anticipados de embarque provienen de múltiples proveedores que manejan diferentes versiones de estándares EDI (como ANSI X12 856 o EDIFACT DESADV):

- **Historial de revisiones embebido:** Si un proveedor modifica las cantidades o cambia un lote mientras el camión está en tránsito, el documento avisos_asn no se sobrescribe a ciegas.
- **Control de retransmisión:**
 - Se registra un contador incremental revision_numero: Integer (iniciando en 1).
 - Se preserva una marca de tiempo fecha_actualizacion: ISODate.
 - Si la modificación altera los lotes o fechas de caducidad, el estado de recepción pasa a REVISADO_EN_TRANSITO y se dispara un evento a alertas_operativas para advertir al personal de andén.

3.7 Estrategia de crecimiento

Para garantizar que la arquitectura soporte el incremento de volumen transaccional, nuevas sucursales y la expansión de modelos predictivos sin degradar latencias, la estrategia de crecimiento se estructura en cuatro dimensiones: particionamiento horizontal, optimización de ciclo de vida, paralelismo en ingesta y expansión analítica.

3.7.1 Escalamiento Horizontal en MongoDB (Sharding)

A medida que el número de tiendas y la frecuencia de inferencias de Machine Learning crezcan, se adopta un esquema de clúster fragmentado.

- **Particionamiento de pronosticos_demanda (Clave Compuesta Hash/Rango):**
 - **Shard Key recomendada:** {"locacion_id": 1, "sku_id": "hashed"}
 - **Justificación:** Agrupa los pronósticos de una misma sucursal en nodos cercanos para agilizar consultas locales de reabastecimiento, distribuyendo uniformemente la carga de escritura entre los fragmentos mediante el hash del SKU para evitar cuellos de botella (*hotspots*).
- **Particionamiento de avisos_asn (Clave por Entidad de Destino):**
 - **Shard Key recomendada:** {"locacion_destino_id": 1, "numero_asn": 1}
 - **Justificación:** Enruta las consultas del personal de muelle directamente al shard que gestiona su centro de distribución o tienda, minimizando operaciones de dispersión-recolección (*scatter-gather*).

3.7.2 Ciclo de Vida del Dato

Para evitar que datos históricos no operativos encarezcan el almacenamiento en memoria (RAM/Working Set), se establece una política de ciclo de vida.

Capa de Datos	Retención	Mecanismo	Acción de Crecimiento
Hot (Operativo)	0 a 30 días	MongoDB (RAM optimizada)	Consultas activas de inferencias vigentes, alertas abiertas y manifiestos en tránsito.
Warm (Análisis Reciente)	31 a 180 días	MongoDB (Almacenamiento estándar)	Índices compactados; solo se consultan para validación de precisión de modelos y auditorías logísticas.
Cold (Histórico)	> 180 días	Data Lake / Object Storage	Un pipeline por lotes exporta predicciones y manifiestos cerrados a almacenamiento

			columnar para reentrenamiento de ML, purgando la base documental.
--	--	--	---

3.7.3 Rendimiento en Ingesta y Paralelismo

Conforme aumenten las lecturas de sensores de frío, radiofrecuencia (RFID) y despachos masivos.

- **Operaciones Masivas Desacopladas (*Bulk Writes*):** La ingesta de inferencias de ML no se realiza documento por documento, sino mediante lotes atómicos utilizando `bulkWrite({ ordered: false })`, permitiendo escrituras paralelas en el motor de almacenamiento WiredTiger.
- **Proyección Estricta en Lecturas:** Los servicios backend y dashboards deben solicitar exclusivamente los campos requeridos mediante proyecciones BSON explícitas (e.g., `{ "prediccion.demanda_esperada_unidades": 1, "_id": 0 }`), reduciendo la sobrecarga de transferencia de red y serialización en memoria.

4. Redis

Redis (*Remote Dictionary Server*) es un motor de estructura de datos en memoria (*in-memory*) de código abierto, diseñado para operar como base de datos no volátil, capa de caché distribuida y gestor de mensajería ligera. A diferencia de PostgreSQL (orientado a persistencia relacional estricta en disco) y MongoDB (diseñado para almacenamiento documental distribuido), Redis ejecuta todas sus lecturas y escrituras directamente en la memoria RAM, ofreciendo tiempos de respuesta predecibles en el orden de los submilisegundos.

En este ecosistema de retail y productos perecederos, Redis cumple un rol estratégico de optimización operativa:

Mitigación de sobrecarga en disco (*Shielding* de PostgreSQL y MongoDB)

Actúa como primera línea de defensa para absorber consultas repetitivas de alta concurrencia —como catálogos maestros de SKUs, validación de sesiones operativas de personal de andén o tokens de autenticación— evitando que el motor transaccional gaste ciclos de I/O en lecturas redundantes.

Manejo de estados volátiles y contadores atómicos

Permite gestionar inventarios temporales en carritos de compra, conteos rápidos de recepciones por radiofrecuencia (RFID) y cuotas de peticiones (*rate limiting*) mediante instrucciones atómicas nativas, garantizando consistencia sin necesidad de bloqueos pesados de tablas.

Gestión de colas de eventos ligeras y pub/sub

Facilita la comunicación desacoplada entre microservicios o pipelines analíticos, notificando en tiempo real cuando se detecta una alerta crítica de caducidad o cuando un nuevo aviso ASN llega al muelle.

4.1 Estructura de Claves

Para evitar colisiones, garantizar predictibilidad en las consultas y facilitar la administración programática de la memoria, Redis adopta una convención de nombrado jerárquico estandarizada mediante separación por dos puntos (:).

La convención base sigue el patrón de diseño:

None

`<dominio>:<entidad>[:<subentidad>:<identificador>[:<atributo>]]`

4.1.1 Catálogo Formal de Claves por Dominio Operativo

Dominio: Catálogos Maestros y Caché de Lectura (cache)

Almacena representaciones serializadas o desglosadas de los catálogos relacionales de PostgreSQL que presentan alta tasa de lectura y baja frecuencia de modificación.

- **Ficha de Producto (SKU):**
 - **Clave:** cache:producto:<sku_id>
 - **Tipo de dato:** Hash
 - **Campos internos:** nombre, categoria_id, temperatura_conservacion, unidad_medida, dias_vida_util
 - **Ejemplo:** cache:producto:1042
 - **Propósito:** Evita consultas recurrentes a operacion.producto en pantallas de punto de venta (POS) y terminales de radiofrecuencia (RFID).
- **Configuración de Sucursal / Locación:**
 - **Clave:** cache:locacion:<locacion_id>
 - **Tipo de dato:** Hash
 - **Campos internos:** nombre, tipo (TIENDA / CEDIS), zona_horaria, capacidad_frio_m3
 - **Ejemplo:** cache:locacion:12

Dominio: Seguridad y Sesión Operativa (session)

Mantiene el estado volátil del personal logístico y administradores autenticados desde terminales móviles de andén o el sistema central.

- **Token de Sesión de Usuario:**

- **Clave:** session:token:<uuid_token>
- **Tipo de dato:** String (JSON serializado con los claims del JWT o identificadores de rol)
- **Ejemplo:** session:token:a8098c1a-f86e-11da-bd1a-00112444be1e
- **Propósito:** Validación de permisos en microsegundos en cada petición HTTP/gRPC sin impactar a la base de datos relacional.
- **Índice de Sesiones Activas por Usuario:**
 - **Clave:** session:user:<usuario_id>:tokens
 - **Tipo de dato:** Set (conjunto de UUIDs de tokens activos)
 - **Ejemplo:** session:user:45:tokens
 - **Propósito:** Permitir invalidación o revocación masiva de sesiones concurrentes de un usuario ante alertas de seguridad.

Dominio: Operación en Tiempo Real y Colas (ops)

Gestiona inventarios temporales, conteos en andén y colas de sincronización asíncrona hacia PostgreSQL o MongoDB.

- **Reserva Temporal de Inventario (Prevención de sobreventa en perecederos):**
 - **Clave:** ops:reserva:lote:<lote_id>
 - **Tipo de dato:** String (valor numérico entero gestionado con INCRBY / DECRBY)
 - **Ejemplo:** ops:reserva:lote:3fa85f64-5717-4562-b3fc-2c963f66afa6
 - **Propósito:** Bloqueo atómico de piezas de un lote específico durante el proceso de armado de pedidos o transferencias entre sucursales.
- **Conteo Rápido de Recepción en Muelle (Recepción ciega vs. ASN):**
 - **Clave:** ops:recepcion:asn:<numero_asn>:sku:<sku_id>
 - **Tipo de dato:** String (contador entero acumulado con INCR)
 - **Ejemplo:** ops:recepcion:asn:ASN-2026-00892:sku:1042
 - **Propósito:** Registro en memoria de cada caja o tarima escaneada por radiofrecuencia en tiempo real antes de consolidar el documento en MongoDB.
- **Cola de Despacho de Eventos hacia MongoDB/PostgreSQL:**
 - **Clave:** ops:queue:asn_confirmados
 - **Tipo de dato:** List (consumida mediante patrones productor-consumidor con LPUSH y BRPOP)
 - **Propósito:** Buffer de amortiguación para evitar picos de escritura concurrentes hacia la base de datos documental cuando se cierran múltiples embarques simultáneos.

4. Dominio: Alertas Rápidas y Priorización (rank)

Estructuras para consultar tableros de incidencias operativas ordenadas por criticidad sin computar ordenamientos en disco.

- **Ranking de Riesgo de Merma por Tienda:**
 - **Clave:** rank:merma_inminente:locacion:<locacion_id>

- **Tipo de dato:** Sorted Set (ZSET)
- **Score:** Nivel de probabilidad de descomposición o días restantes de vida útil ponderados.
- **Member:** <sku_id>
- **Ejemplo:** rank:merma_inminente:locacion:12
- **Propósito:** Permite a la aplicación obtener instantáneamente los 10 productos con mayor riesgo de pérdida mediante ZREVRANGEBYSCORE en tiempo $O(\log N + M)$.

4.2 Tiempo de expiración

En un entorno en memoria de alto rendimiento, la asignación explícita de tiempos de expiración (*Time-To-Live* o TTL) es obligatoria para evitar fugas de memoria, garantizar la frescura de datos de negocio y mantener el consumo de RAM dentro de los umbrales operativos.

Clave / Patrón	Estructura	TTL Nominal	Equivalencia Temporal	Justificación de Negocio y Operativa
cache:producto:<sku_id>	Hash	86400 seg	24 horas	El maestro de productos rara vez cambia durante la jornada operativa. Si ocurre una modificación manual en PostgreSQL, el backend ejecuta una invalidación activa (DEL).
cache:locacion:<locacion_id>	Hash	604800 seg	7 días	Parámetros de sucursales y CEDIS son estáticos; una ventana de una semana reduce a cero las lecturas a base de datos relacional para metadatos de tienda.
session:token:<token>	String	28800 seg	8 horas	Corresponde exactamente a un turno laboral estándar de los operadores de muelle y personal de tienda, cerrando accesos automáticamente por seguridad.
session:user:<user_id>:tokens	Set	28800 seg	8 horas	Sincronizado con la vigencia máxima del turno; se renueva

				o elimina al cerrar la sesión operativa.
ops:reserva:lote:<lote_id>	String	900 seg	15 minutos	Tiempo máximo asignado a una terminal móvil o caja para consolidar un movimiento de mercancía. Si la transacción no se confirma, el bloqueo expira y el stock se libera automáticamente.
ops:recepcion:asn:<asn> :sku:<sku>	String	43200 seg	12 horas	Margen suficiente para que el camión descargue por completo en andén. Tras el cierre del manifiesto y la persistencia en MongoDB, la clave se borra explícitamente.
ops:queue:asn_confirmados	List	<i>Sin TTL directo</i>	N/A (Transitorio)	Se gestiona bajo patrón productor-consumidor. La persistencia en disco de Redis (AOF) garantiza que la cola no se pierda; se vacía conforme los <i>workers</i> procesan los mensajes.
rank:merma_inminente:locacion:<loc>	Sorted Set	3600 seg	1 hora	Las predicciones de descomposición y cálculos FEFO se recalculan por lote cada hora; el TTL garantiza que nunca se desplieguen tableros con inferencias obsoletas.

4.3 Sesiones

En entornos logísticos de alta rotación (andenes de CEDIS, cámaras frigoríficas y puntos de venta), la validación de identidad no puede depender de consultas directas a PostgreSQL en cada petición HTTP o gRPC. Redis actúa como el almacén centralizado de sesiones de baja latencia, desacoplando la carga transaccional del esquema relacional y garantizando control de concurrencia y revocación inmediata de accesos.

4.3.1 Modelo de Datos de Sesión

Para cada operador o supervisor autenticado se utilizan dos estructuras complementarias:

- **Token de Acceso Principal (Ficha de Sesión):**
 - **Clave:** session:token:<uuid_token>
 - **Tipo:** String (JSON serializado con payload comprimido) o Hash
 - **Estructura interna de datos:**
 - JSON

```
None
{
  "usuario_id": 45,
  "username": "slopez",
  "rol": "OPERADOR_ANDEN",
  "locacion_id": 12,
  "dispositivo_ip": "192.168.10.150",
  "terminal_id": "RFID-HANDHELD-04",
  "autenticado_en": "2026-09-08T08:30:00Z"
}
```

-
- **TTL:** 28800 segundos (8 horas nominales).
- **Índice Invertido de Concurrencia por Usuario:**
 - **Clave:** session:user:<usuario_id>:tokens
 - **Tipo:** Set
 - **Contenido:** Conjunto de UUIDs de tokens activos emitidos para ese usuario_id.
 - **Propósito:** Permite auditar qué dispositivos tiene abiertos un mismo colaborador y habilitar políticas de desconexión remota.

4.3.2 Flujo Operativo del Ciclo de Vida de Sesión

A. Inicio de Sesión (Login)

1. El usuario se autentica en el backend contra la tabla operacion.usuario de PostgreSQL (validando hash de contraseña con crypt()).
2. El backend genera un identificador de sesión único (UUIDv4).
3. Se persisten atómicamente en Redis la ficha de sesión y la referencia en el conjunto del usuario usando un pipeline/transacción:

```
Shell
SET session:token:c2e1b6f0-8c29-4d1a-a5f1-39bb619bfa01
'{"usuario_id":45,"rol":"OPERADOR_ANDEN","locacion_id":12}' EX 28800
SADD session:user:45:tokens "c2e1b6f0-8c29-4d1a-a5f1-39bb619bfa01"
EXPIRE session:user:45:tokens 28800
```

B. Autorización en Cada Petición

1. La terminal móvil envía el token en la cabecera Authorization: Bearer <uuid_token>.
2. El API Gateway o middleware del backend ejecuta:

```
Shell
GET session:token:<uuid_token>
```

3. Si la clave existe, deserializa el contexto (rol, locacion_id) y autoriza la acción sin tocar PostgreSQL.
4. **Ventana deslizante (*Sliding Window*)**: Si la clave sigue vigente, renueva el TTL para mantener activa la sesión mientras el operador continúe trabajando:

```
Shell
EXPIRE session:token:<uuid_token> 28800
```

C. Cierre de Sesión (Logout)

1. Al salir de la aplicación en la terminal:

```
Shell
DEL session:token:<uuid_token>
SREM session:user:45:tokens "<uuid_token>"
```

4.4 Lista de Tokens Revocados en Memoria

Cuando la arquitectura de seguridad utiliza tokens autónomos o emitidos con firma digital (como JWTs), estos no pueden destruirse físicamente en el cliente hasta que expire su tiempo de vida natural (exp). Para permitir cierres de sesión inmediatos, desconexiones forzadas por cambio de credenciales o bloqueo de terminales móviles extraviadas en

andenes de distribución, se implementa una **lista de revocación en memoria** dentro de Redis.

4.4.1 Flujo Operativo de Validación y Revocación

A. Operación de Revocación (Logout / Forzar Cierre)

Cuando un operador presiona "Cerrar Sesión" en su terminal de radiofrecuencia (RFID) o un administrador cancela el acceso a un empleado en PostgreSQL:

1. El backend extrae el campo jti y el claim de expiración exp del token presentado.
2. Calcula la diferencia temporal restante en segundos (Δt).
3. Inyecta la marca en Redis:
4. Bash

None

```
SET auth:revoked:8a9b2c3d-4e5f-6a7b-8c9d-0e1f2a3b4c5d "1" EX 7200
```

B. Interceptación en el API Gateway / Middleware

En cada petición entrante hacia los servicios de la plataforma:

1. El middleware valida la firma criptográfica y la vigencia temporal estándar del token ($O(1)$ en CPU).
2. Si el token es matemáticamente válido, consulta en Redis si está listado como revocado:

Shell

```
EXISTS auth:revoked:8a9b2c3d-4e5f-6a7b-8c9d-0e1f2a3b4c5d
```

3. Si el comando retorna 1, la solicitud es rechazada inmediatamente con un código HTTP 401 Unauthorized (Token revocado o sesión finalizada).
4. Si retorna 0, el flujo continúa hacia el controlador de negocio. El impacto en latencia es inferior a 0.5.

4.4.2 Escenarios Críticos de Revocación

- Revocación Masiva por Empleado (Invalidación de Credenciales)
 - Si se modifica la contraseña de un usuario o se cambia su rol en la tabla operacion.usuario, el sistema debe invalidar todos sus dispositivos activos.
 - Para evitar registrar decenas de tokens individuales en la denylist, se almacena una marca temporal de corte por usuario:
 - **Clave:** auth:user:invalid_before:<usuario_id>
 - **Tipo:** String (Unix Timestamp en segundos)
 - **Ejemplo:** auth:user:invalid_before:45 = 1757328000

- El middleware compara el claim de emisión (iat - *Issued At*) del token contra este valor: cualquier token emitido antes de esa marca temporal es rechazado de inmediato.
- **Revocación por Pérdida de Terminal en Almacén**
 - Si una terminal física se extravía en el CEDIS, el supervisor revoca el identificador del dispositivo (device_id).
 - Se registra auth:revoked:device:<device_id> con un TTL equivalente a la sesión máxima (8 horas), bloqueando cualquier tráfico que provenga de ese hardware.

4.5 Caché de Consultas

La caché de consultas en Redis desacopla el tráfico de lectura repetitiva hacia PostgreSQL y MongoDB, interceptando solicitudes idénticas para responderlas en microsegundos desde memoria RAM. Aplica el patrón **Cache-Aside**, reduciendo el costo de agregaciones pesadas, cálculos de política FEFO y proyecciones analíticas de alta concurrencia.

4.5.1 Consultas Prioritarias en Caché

- **Inventario Consolidado por Tienda y SKU:**
 - **Clave:** query:stock:locacion:<loc_id>sku:<sku_id>
 - **Tipo:** String (JSON serializado con total disponible, piezas comprometidas y detalle de lotes próximos a vencer).
 - **TTL:** 300 seg (5 minutos).
 - **Alcance:** Evita recalcular sumatorias relacionales (SUM(cantidad)) en operacion.existencia cada vez que el punto de venta o la terminal RFID consulta disponibilidad.
- **Top de Mermas y Riesgo FEFO por Sucursal:**
 - **Clave:** query:fefo:riesgo:locacion:<loc_id>
 - **Tipo:** String (JSON serializado con los lotes en ventana crítica de caducidad).
 - **TTL:** 900 seg (15 minutos).
 - **Alcance:** Almacena el resultado del cruce entre existencias físicas y la colección pronosticos_demanda, alimentando los paneles de supervisores sin reejecutar filtros analíticos complejos.
- **Filtros de Catálogo Maestro:**
 - **Clave:** query:catalogo:activos:categoria:<categoria_id>
 - **Tipo:** String (Lista de SKUs activos y metadatos base).
 - **TTL:** 14400 seg (4 horas).
 - **Alcance:** Atiende las búsquedas masivas de productos en cajas registradoras y sistemas de pedidos sin tocar la base relacional.

4.5.2 Ciclo Operativo (Cache-Aside)

1. El backend recibe la petición y consulta la clave calculada en Redis.

2. **Cache Hit:** Si la clave existe, devuelve la respuesta de inmediato sin tocar el disco.
3. **Cache Miss:** Si expira o no existe, ejecuta la consulta en PostgreSQL o MongoDB, serializa el resultado en JSON, lo persiste en Redis con su TTL y lo entrega al cliente.
4. **Invalidación por Evento:** Operaciones transaccionales críticas (como el cierre de una venta en `operacion.venta_detalle` o la recepción de un lote) ejecutan un DEL explícito sobre la clave correspondiente para forzar su rehidratación fresca en la siguiente lectura.

4.6 Contadores

Los contadores en Redis gestionan métricas de alta frecuencia y control de concurrencia directamente en memoria principal. Gracias a operaciones atómicas nativas (INCR, INCRBY, DECRBY), resuelven incrementos concurrentes en tiempo constante $O(1)$ sin generar condiciones de carrera (*race conditions*) ni provocar bloqueos de tablas o filas en PostgreSQL.

4.6.1 Casos de Uso Críticos en el Proyecto

- **Conteo Rápido de Descarga en Muelle (Recepción Ciega ASN)**
 - **Clave:** `count:asn:<numero_asn>:sku:<sku_id>:recibido`
 - **Tipo:** String (Entero de 64 bits)
 - **Operación:** INCR o INCRBY `<bultos>` por cada escaneo de pistola RFID / código de barras.
 - **TTL:** 43200 seg (12 horas, sincronizado con el tiempo de descarga).
 - **Propósito:** Registrar bultos físicos recibidos en tiempo real. Al finalizar la maniobra, el total acumulado se compara en memoria contra `avisos_asn.detalle_carga.cantidad_embarcada` en MongoDB; si coinciden, se consolida la recepción sin saturar la base relacional con miles de escrituras individuales.
- **Reserva Atómica de Stock Transitorio**
 - **Clave:** `count:reserva:lote:<lote_id>`
 - **Tipo:** String (Entero)
 - **Operación:** INCRBY `<cantidad>` al apartar piezas para un pedido; DECRBY `<cantidad>` si la transacción se cancela o aborta.
 - **TTL:** 900 seg (15 minutos).
 - **Propósito:** Prevenir sobreventa (*overselling*) de lotes perecederos durante la preparación de pedidos concurrentes sin colocar bloqueos pesados (SELECT ... FOR UPDATE) en `operacion.existencia`.
- **Métricas Operativas Diarias (Auditoría Volátil)**
 - **Clave:** `count:metricas:locacion:<loc_id>:fecha:<YYYYMMDD>:<evento>`
 - **Sub-eventos:** `:ventas`, `:mermas_descarte`, `:alertas_criticas`
 - **Tipo:** String (Entero)
 - **Operación:** INCR disparado tras cada evento operativo.

- **TTL:** 172800 seg (48 horas).
- **Propósito:** Alimenta los tacómetros y gráficas de actividad en tiempo real de los tableros gerenciales. Un proceso en segundo plano (*worker*) lee estas métricas acumuladas y las persiste consolidadas en el esquema relacional al final de la jornada.
- **Limitador de Tasa de Peticiones (*Rate Limiting* en API Gateway)**
 - **Clave:** rate:ip:<direccion_ip>:minuto:<timestamp_minuto>
 - **Tipo:** String (Entero)
 - **Operación:** INCR + EXPIRE 60 en la primera llamada.
 - **Propósito:** Proteger las APIs de inventario y pedidos contra ráfagas excesivas de consultas o ataques de denegación de servicio desde terminales móviles o servicios externos.

4.7 Bloqueos distribuidos

En una arquitectura con múltiples contenedores de aplicación, microservicios concurrentes y trabajadores en segundo plano (*workers*), los bloqueos a nivel de hilo o de proceso local no son suficientes. Redis proporciona un mecanismo de **bloqueo distribuido** (*Distributed Lock*) centralizado y de baja latencia para coordinar el acceso exclusivo a recursos críticos sin recurrir a bloqueos pesados de tablas o filas en PostgreSQL (SELECT ... FOR UPDATE), evitando contención de I/O y riesgo de interbloqueos (*deadlocks*).

4.7.1 Casos de Uso Críticos en el Proyecto

- **Consolidación de Manifiestos de Recepción (Andén/CEDIS):**
 - **Recurso protegido:** Cierre simultáneo del manifiesto avisos_asn.
 - **Problema:** Dos supervisores en muelles distintos podrían intentar confirmar la recepción o reconciliar inventario del mismo embarque al mismo tiempo, duplicando las existencias creadas en PostgreSQL.
- **Asignación de Lote Perecedero Crítico (Política FEFO):**
 - **Recurso protegido:** Lote específico en operacion.lote con muy pocas unidades y vida útil inmediata.
 - **Problema:** Múltiples pedidos de ecommerce o traslados de tienda compitiendo en paralelo por agotar el último saldo de un lote próximo a caducar.
- **Disparo del Pipeline de Inferencia de Demanda:**
 - **Recurso protegido:** Ejecución del job analítico para un locacion_id y sku_id.
 - **Problema:** Tareas programadas concurrentes o peticiones manuales desde tableros disparando el mismo modelo de ML en paralelo, saturando CPU y generando escrituras redundantes en MongoDB.

4.7.2 Patrón de Implementación y Estructura de Claves

Para garantizar seguridad frente a fallos y evitar bloqueos permanentes si un proceso colapsa, se implementa el patrón canónico con las primitivas atómicas de Redis:

- **Convención de clave de bloqueo:**
 - **Clave:** lock:<dominio>:<recurso_id>
 - **Valor:** <uuid_propietario> (un identificador aleatorio único por hilo/transacción).
 - **Ejemplo:** lock:asn:ASN-2026-00892 o lock:lote:3fa85f64-5717-4562-b3fc-2c963f66afa6

4.7.3 Ciclo de Vida del Bloqueo

1. Adquisición Atómica (SET NX PX)

El proceso intenta adquirir el recurso en una única instrucción con tiempo de expiración forzado (*lease time*):

Shell

```
SET lock:asn:ASN-2026-00892 "a9f8b4c2-0192-4fbc-bdf1-e374d82b1234" NX PX 10000
```

- - **NX (*Not eXists*):** Asigna el valor únicamente si la clave no existe previamente. Si otro hilo ya tiene el candado, el comando retorna nil de inmediato sin bloquear hilos del sistema.
 - **PX 10000:** Asigna un TTL estricto en milisegundos (10 segundos). Si el proceso sufre un fallo de hardware o la red se desconecta antes de liberar el candado, Redis lo expira automáticamente, previniendo un bloqueo eterno (*starvation*).

2. Liberación Segura con Script Lua

Un cliente jamás debe ejecutar un simple DEL para liberar el bloqueo; si la operación tardó más tiempo que el TTL asignado, el candado original podría haber expirado y haber sido adquirido por otro proceso. Un DEL ciego destruiría el bloqueo del segundo proceso.

La liberación se condiciona mediante un script Lua atómico que verifica que el token del poseedor coincida con el registrado.

Textproto

```
if redis.call("get", KEYS[1]) == ARGV[1] then
    return redis.call("del", KEYS[1])
else
    return 0
end
```

- **KEYS[1]:** Clave del bloqueo (lock:asn:ASN-2026-00892).

- **ARGV[1]:** Identificador único generado por el proceso (a9f8b4c2-0192-4fbc-bdf1-e374d82b1234).

4.7.4 Parámetros de Configuración

Tipo de Proceso	Clave de Bloqueo	TTL / Expiración (PX)	Estrategia de Reintento
Cierre y consolidación de ASN	lock:asn:<numero_asn>	10000 ms (10 seg)	Fallo rápido (<i>fail-fast</i>) informando al operador que la orden ya está en proceso de cierre.
Reserva/descuento de lote FEFO	lock:lote:<lote_id>	3000 ms (3 seg)	Reintentos exponenciales (<i>exponential backoff</i>) con jitter (3 intentos, margen de 50 a 150 ms).
Corrida de modelo ML	lock:ml:locacion:<loc_id>:sku:<sku_id>	60000 ms (60 seg)	Descarte inmediato si ya existe una corrida activa para ese corte horario.

4.8 Datos Temporales

Los datos temporales corresponden a eventos de alta frecuencia y ciclo de vida corto generados por la operación logística, tales como telemetría de frío, marcas de paso por antenas RFID y auditoría de eventos en tránsito. Persistir cada una de estas lecturas individuales de forma inmediata en PostgreSQL o MongoDB generaría saturación innecesaria de I/O en disco. Redis absorbe este flujo masivo de escrituras en memoria, sirviendo como zona de aterrizaje transitoria antes de su consolidación o descarte.

4.8.1 Casos de Uso y Estructuras de Datos

- **Monitoreo Efímero de Cadena de Frío (Telemetría de Transporte)**
 - **Clave:** temp:asn:<numero_asn>:sensor:<sensor_id>
 - **Tipo:** Sorted Set (ZSET)
 - **Score:** Marca de tiempo Unix en segundos (timestamp).
 - **Member:** Lectura de temperatura serializada (ej. "-18.4" o "-18.4:bateria_ok").
 - **TTL:** 86400 seg (24 horas).
 - **Propósito:** Registrar las lecturas de los termógrafos de las cajas refrigeradas durante el trayecto. Al llegar al andén, el supervisor consulta el rango temporal completo con ZRANGEBYSCORE para validar que nunca se rompiera la cadena de frío, resumiendo el valor mínimo y máximo en el documento final de avisos_asn sin guardar miles de lecturas individuales en MongoDB.
- **Trazabilidad de Paso de Tarimas en Andén (Lecturas RFID)**
 - **Clave:** stream:rfid:locacion:<locacion_id>:puerta:<puerta_id>
 - **Tipo:** Stream (o List)
 - **Entrada:** XADD stream:rfid:locacion:12:puerta:3 * epc "3034257BF400B" lote "LOTE-098" ts "1757328000"
 - **TTL / Retención:** MAXLEN ~ 10000 (conserva solo los últimos 10,000 eventos de paso).
 - **Propósito:** Captura ráfagas continuas de cajas pasando bajo el arco lector del muelle, permitiendo a los procesos de descarga validar el inventario en paralelo sin cuellos de botella.
- **Bandeja de Eventos Volátiles de Alerta (Notificaciones Push)**
 - **Clave:** temp:notificaciones:locacion:<locacion_id>
 - **Tipo:** List (manejada con LPUSH y LTRIM)
 - **Regla de retención:** Se limita a los últimos 50 eventos (LTRIM 0 49).
 - **TTL:** 43200 seg (12 horas).
 - **Propósito:** Alimenta la campana de notificaciones inmediatas en las terminales móviles de los operadores sin requerir una consulta a la colección alertas_operativas.

4.8.2 Ciclo Operativo: Captura, Consolidación y Purga (Buffer & Flush)

Ingesta continua

Los dispositivos periféricos escriben directamente en estructuras de memoria (ZADD, XADD) con latencia inferior a 1 ms.

Consultas de ventana

El sistema realiza lecturas analíticas sobre ventanas de tiempo concretas (e.g., últimas 2 horas de temperatura) utilizando rangos numéricos sobre el Score del Sorted Set.

Consolidación programada

Al cerrar el proceso operativo (e.g., camión descargado y validado), un worker calcula únicamente los estadísticos relevantes (temperatura mínima registrada, máxima y promedio) y los adjunta al manifiesto definitivo.

Purga automática

Las lecturas atómicas de segundo a segundo no se archivan en bases de datos relacionales; expiran y se liberan de memoria RAM de forma autónoma gracias al TTL de la clave.

5. Conclusiones

La arquitectura de datos diseñada consolida un ecosistema políglota equilibrado, donde cada motor asume la carga para la que fue específicamente concebido. PostgreSQL garantiza la integridad referencial y financiera de las transacciones operativas mediante tipado estricto, esquemas normalizados y auditoría inmutable en JSONB. En paralelo, MongoDB aporta la flexibilidad necesaria para absorber inferencias predictivas de Machine Learning y manifiestos logísticos heterogéneos, recurriendo al anidamiento jerárquico para optimizar lecturas atómicas sin penalizar el rendimiento del motor transaccional. Por último, Redis opera como el escudo perimetral de ultra baja latencia, absorbiendo ráfagas de lectura mediante caché de consultas, coordinando la concurrencia distribuida en andenes y desacoplando la persistencia de estados efímeros como sesiones, telemetría y contadores.

Bajo este modelo, la infraestructura se rige bajo principios estrictos de Infrastructure as Code (IaC), permitiendo que el despliegue local, en entornos de desarrollo o sobre máquinas virtuales en la nube se ejecute de manera determinista, repetible y sin dependencias cruzadas mediante Docker Compose y automatización en Makefiles. Esta cohesión técnica no solo resuelve los cuellos de botella clásicos en la gestión de productos perecederos —como la caducidad FEFO, la rotación en frío y las alertas tempranas de desabasto—, sino que establece una base elástica, gobernable y preparada para escalar horizontalmente conforme aumente el volumen operativo y la red logística de la empresa.